

Website Development Pre-Launch Checklist

A practical end-to-end checklist for planning, designing, developing, testing, securing, optimizing, and launching modern websites.

Created by @developer_lokesh

Instagram: https://instagram.com/developer_lokesh

How to use this checklist

Mark each item Done, Not Applicable, or Needs Review. The checklist should scale with the project: a landing page needs less process than an e-commerce platform or SaaS product.

1. Discovery & Requirements

- Define the business/product goal and primary user action.
- Identify target users, personas, pain points, and use cases.
- Define measurable success criteria/KPIs.
- Research competitors and relevant UX patterns.
- List must-have features, nice-to-have features, and explicit non-goals.
- Confirm timeline, budget, stakeholders, approvals, and content ownership.

2. Information Architecture & UX

- Create a sitemap and navigation hierarchy.
- Map critical user journeys and conversion flows.
- Define all required pages and states: loading, empty, success, error, 404.
- Define URL structure and naming conventions.
- Plan forms, validation, confirmations, and recovery paths.
- Consider mobile, touch, keyboard, and screen-reader interactions.

3. UI / Visual Design

- Establish typography, spacing, colors, borders, radii, shadows, and component states.
- Build reusable UI components/design tokens.
- Design responsive layouts for mobile, tablet, laptop, and desktop.
- Prepare optimized logos, images, icons, illustrations, and other assets.
- Check contrast, focus states, readable typography, and motion behavior.

4. Technical Planning

- Choose frontend framework/library, styling strategy, and component approach.
- Choose backend architecture, database, API style, and authentication strategy if required.
- Decide hosting, deployment, storage/CDN, and monitoring approach.
- Define repository, branching, pull-request, code-review, and release workflow.
- Separate development/staging/production environments when appropriate.
- Document major architectural decisions.

5. Frontend Development

- Use a consistent folder/module structure appropriate to project size.
- Keep components reusable and responsibilities clear.
- Separate presentation, business logic, API services, and utilities where practical.
- Handle loading, empty, success, error, and offline states.
- Validate forms on the client for UX and on the server for security.
- Remove dead code, debug logs, unnecessary dependencies, and unused assets.

6. Backend, API & Database

- Define endpoints, HTTP methods, request/response formats, and status codes.
- Validate body, query, and route parameters.
- Implement authentication and authorization where needed.
- Use consistent API error responses.
- Add pagination/filtering for large datasets.
- Define database schemas, relationships, indexes, migrations, and backup strategy.
- Never expose database credentials or private secrets to the client.

7. Security

- Use HTTPS in production.
- Keep dependencies updated and scan for vulnerabilities.
- Protect against XSS, injection, broken access control, and CSRF where applicable.
- Sanitize/encode untrusted content.
- Hash passwords using a modern password-hashing algorithm.
- Configure cookies/tokens securely.
- Configure CORS intentionally.
- Use appropriate security headers and rate limiting.
- Validate file uploads by type and size.
- Never commit secrets, API keys, or production credentials.

8. Performance

- Optimize image dimensions, formats, compression, and responsive delivery.
- Lazy-load non-critical media and heavy features.
- Minimize JavaScript and code-split large features when useful.
- Reduce render-blocking resources.
- Cache static assets appropriately.
- Check slow API requests and database queries.
- Test on slower networks and lower-powered mobile devices.
- Measure Core Web Vitals/performance rather than relying only on a fast development machine.

9. SEO

- Give each indexable page a unique title and useful meta description.
- Use semantic HTML and a logical heading hierarchy.
- Use descriptive, crawlable URLs.
- Add meaningful alt text to informative images.
- Add internal links where useful.
- Configure sitemap.xml and robots.txt intentionally.
- Check canonical URLs, redirects, and noindex rules.
- Add structured data where genuinely useful.
- Verify Open Graph/social sharing metadata.

10. Accessibility

- Make important functionality keyboard accessible.
- Provide visible focus indicators.

- Use semantic HTML and accessible labels.
- Maintain sufficient color contrast.
- Do not communicate meaning through color alone.
- Provide meaningful alternative text.
- Respect reduced-motion preferences where appropriate.
- Test critical flows with a screen reader.

11. Testing & QA

- Test every primary user flow.
- Test valid, invalid, missing, and boundary inputs.
- Test authentication, permissions, logout, and recovery where applicable.
- Test API failures, timeouts, empty data, and server errors.
- Test 404s, deep links, refresh behavior, and redirects.
- Test relevant browsers and viewport sizes.
- Test touch interactions and slow networks.
- Run automated tests and regression checks.
- Verify the production build in staging before release.

12. Analytics & Monitoring

- Define important events and conversions.
- Verify analytics events before launch.
- Configure privacy/consent requirements where applicable.
- Set up uptime monitoring for important production sites.
- Track frontend/backend errors and performance.
- Define alert ownership and incident response steps.

13. Deployment & Launch

- Confirm production environment variables and secret management.
- Confirm domain, DNS, and SSL.
- Confirm database, storage, CDN, and third-party service configuration.
- Confirm build/deployment pipeline.
- Confirm backups and rollback strategy.
- Verify final SEO, metadata, favicon, social preview, forms, emails, and payments if applicable.
- Run production smoke tests immediately after deployment.
- Monitor errors, uptime, analytics, and critical journeys after launch.

14. Final One-Page Gate

- Requirements approved ■ Design approved ■ Critical bugs resolved
- Responsive verified ■ Accessibility reviewed ■ Security reviewed
- Performance checked ■ SEO checked ■ Analytics verified
- Production configuration verified ■ Backup/rollback ready ■ Smoke test passed

Recommended Workflow

Discovery → Requirements → Sitemap → User Flows → Wireframes → UI Design → Technical Planning → Development → Testing → Security Review → Performance/SEO Review → Staging → Launch → Monitoring → Iteration

Key Principle

Do not over-engineer a small website. A good development process is proportional to risk, complexity, team size, and expected growth. The goal is a maintainable, secure, accessible, fast, and useful website—not a checklist with every box mechanically ticked.

Developer Resource by @developer_lokesh

Instagram: https://instagram.com/developer_lokesh